Évaluation – Réseaux et programmation dynamique

Terminale NSI – Durée : 2 heures

Exercice 1 : Réseaux informatiques et protocoles de routage (10 points)

Cet exercice porte sur les **représentations binaires** et les **protocoles de routage**.

1. Adressage IPv4

Une adresse IPv4 est représentée sous la forme de **4 nombres séparés par des points**. Chacun de ces nombres peut être représenté sur un **octet**.

a. Donner en écriture décimale l'adresse IPv4 correspondant à l'écriture binaire :

```
11000000.10101000.10000000.10000011
```

b. Tous les ordinateurs du réseau A ont une adresse IPv4 de la forme : `192.168.128.___` , où seul le dernier octet diffère.

Donner le nombre d'adresses différentes possibles du réseau A.

2. Protocole RIP

On rappelle que le protocole **RIP** cherche à minimiser le **nombre de routeurs traversés**, ce qui correspond à la métrique.

On donne les tables de routage d'un réseau informatique composé de 5 routeurs (appelés A, B, C, D et E), chacun associé directement à un réseau du même nom, obtenues avec le protocole RIP :

Routeur A

Destination	Métrique
A	0
B	1
C	1
D	1
E	2

Routeur B

Destination	Métrique
A	1
B	0
C	2
D	1
E	2

Routeur C

Destination	Métrique
A	1
B	2
C	0
D	1
E	2

Routeur D

Destination	Métrique
A	1
B	1
C	1
D	0
E	1

Routeur E

Destination	Métrique
A	2
B	2
C	2
D	1
E	0

a. Donner la liste des routeurs avec lesquels le routeur A est directement relié.

b. Représenter graphiquement et de manière sommaire les 5 routeurs ainsi que les liaisons existantes entre ceux-ci.

3. Protocole OSPF

Le protocole **OSPF** est un protocole de routage qui cherche à minimiser la **somme des métriques des liaisons** entre routeurs.

Dans le protocole OSPF, le débit des liaisons entre routeurs agit sur la métrique via la relation :

$$\textbf{métrique} = 10^8 / \textbf{débit}$$

dans laquelle le débit est exprimé en **bit par seconde (bps)**.

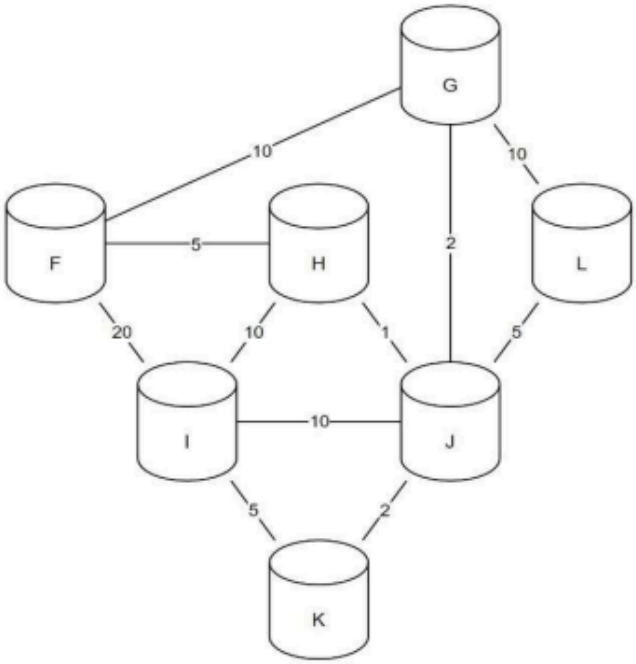
On rappelle qu'un **kbps** est égal à 10^3 **bps** et qu'un **Mbps** est égal à 10^6 **bps**.

Recopier sur votre copie et compléter le tableau suivant :

Débit	100 kbps	500 kbps	?	100 Mbps
Métrique associée	1 000	?	10	1

4. Routage avec OSPF

Voici la représentation d'un réseau et la table de routage incomplète du routeur F obtenue avec le protocole OSPF :



Routeur F

Destination	Métrique
F	0
G	8
H	5
I	
J	
K	
L	

a. Indiquer le chemin emprunté par un message d'un ordinateur du réseau F à destination d'un ordinateur du réseau I. Justifier votre réponse.

b. Recopier et compléter la table de routage du routeur F.

c. Citer une unique panne qui suffirait à ce que toutes les données des échanges de tout autre réseau à destination du réseau F transitent par le routeur G. Expliquer en détail votre réponse.

Exercice 2 : Récursivité et programmation dynamique (10 points)

L'entreprise **Sun and Steel** produit et vend des barres métalliques de différentes longueurs dont voici les prix de vente (en fonction des longueurs) :

Longueur (m)	1	2	3	4	5	6	7	8	9	10
Prix (euros)	1	5	8	9	10	17	17	20	24	30

Quand l'entreprise produit une barre de longueur **n**, elle a deux possibilités :

- vendre la barre telle quelle ;
- la découper en plusieurs parties et les vendre.

Par exemple, pour une barre de 3 mètres, l'entreprise peut agir ainsi :

- vendre la barre de 3 mètres : 8 euros de revenus ;
- produire une barre de 2 mètres et une barre d'un mètre : 5 + 1, soit 6 euros de revenus ;
- produire trois barres de 1 mètre chacune : 3 × 1, soit 3 euros de revenus.

Question 1

Donner la liste de toutes les possibilités pour l'entreprise lors de la vente d'une **barre de 4 mètres**. Quel est le **revenu optimal** ?

Dans la suite du problème, une barre mesure désormais **n** mètres, avec **n** compris entre 1 et 10 mètres.

- Pour chaque longueur, on détermine ainsi le **revenu maximal** possible, noté **r(n)** ; ainsi, **r(3)** vaut 8 euros par exemple.
- Le prix de chaque barre métallique (non découpée) est noté **p(n)** ; ainsi, **p(3)** vaut 8 euros.

L'idée est d'écrire un programme en Python permettant de calculer le revenu maximal engendré pour chaque longueur de barre métallique.

On peut remarquer qu'il est aussi possible de calculer **r(5)** de la méthode suivante :

r(5) = max(p(0) + r(5), p(1) + r(4), p(2) + r(3), p(3) + r(2), p(4) + r(1), p(5) + r(0)) avec p(0) et r(0) valant zéro euro puisqu'il s'agit de barres métalliques de longueur nulle.

On peut généraliser au rang **n** avec la relation suivante :

r(n) = max(p(0) + r(n), p(1) + r(n-1), p(2) + r(n-2), … , p(n-1) + r(1), p(n) + r(0))

Question 2

En s'aidant des informations ci-dessus, compléter (sur la copie) la fonction `revenu_barre(n)` .

```
def revenu_barre(n):
    # Les indices de prix représentent la longueur
    # de la barre.
    prix = [0,1,5,8,9,10,17,17,20,24,30]

    # Cas d'arrêt
    if n == 0:
        return ...

    # Cas général
    else:
        revenu = 0
        for i in range(1, n+1):
            revenu = max(revenu, prix[i] + ...)

    return revenu

# Jeu de test
print(revenu_barre(10))  # Attendu : 30
```

Question 3

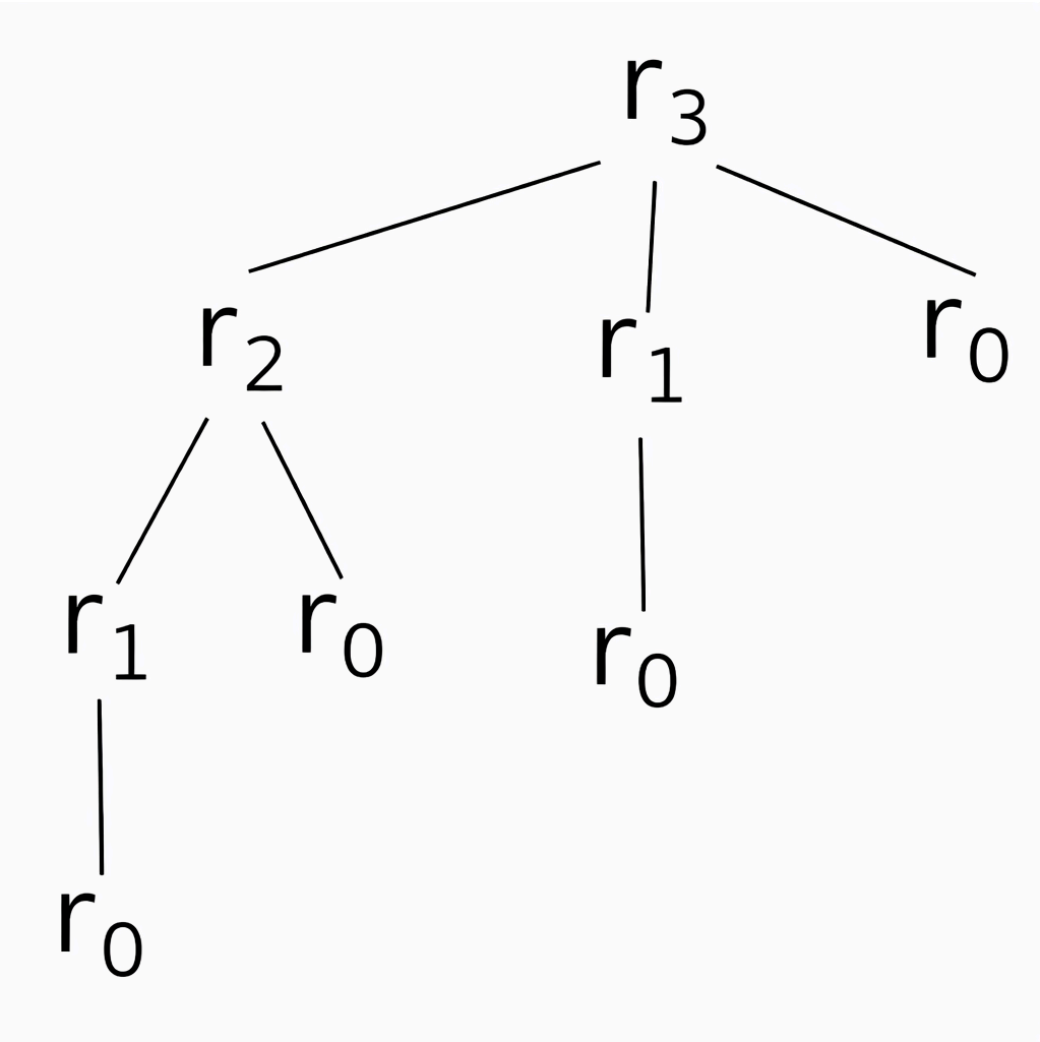
L'entreprise a modifié ses tarifs :

Longueur (m)	1	2	3	4	5	6	7	8	9	10
Prix (euros)	1	6	9	11	12	19	20	23	24	26

Que doit-on changer dans le programme ci-dessus pour qu'il corresponde aux nouveaux tarifs ?

Question 4

Voici l'arbre d'appel récursif du calcul de **r(3)** :



1/ Combien de fois est calculé **r1** ? Quel type de problèmes cela pourra-t-il poser ?

2/ Dessiner l'arbre d'appels récursifs de **r(4)**.

Une approche descendante …

Question 5

Proposer alors une version de la fonction précédente en stockant les valeurs déjà calculées dans un dictionnaire (mémorisation) pour corriger les problèmes précédents : on recopiera et complétera le programme sur la copie.

Une approche ascendante …

Question 6

Compléter le programme suivant, version non récursive de la précédente :

```
def revenu_barre(n):
    # Les indices de prix représentent la longueur
    # de la barre.
    prix = [0,1,5,8,9,10,17,17,20,24,30]
    # Stocke les revenus optimisés
    revenu = [0]*11

    for i in range(1, n+1):
        for j in range(...):
            revenu[i] = max(revenu[i], prix[j] + ...)

    return ...

# Jeu de test
print(revenu_barre(5))  # Attendu : 13
```